

LAB 3: Inverse & "Pseudoinverse" of a Matrix

[see also James (1978) posted on WEBSITE]

⇒ MANY LINEAR SYSTEMS REDUCE TO PROBLEMS OF THE FORM

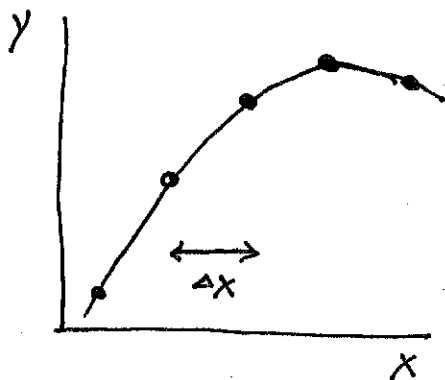
$$\begin{pmatrix} A_{11} & A_{12} & \cdot & \cdot & \cdot \\ A_{21} & A_{22} & \cdot & \cdot & \cdot \\ \vdots & \vdots & & & \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \end{pmatrix}$$

$$\tilde{A} \tilde{X} = \tilde{Y}$$

Solve by inverting matrix $(\tilde{A}^{-1})(\tilde{A}) = \mathbb{I}$ "Identity" Matrix

$$\tilde{X} = \tilde{A}^{-1} \tilde{Y} \quad \dots \text{so long as inverse exists}$$

⇒ Ex: Polynomial Interpolation



- * Know $y_i(x_i)$ at discrete points
- * Want approximation for arbitrary $y(x)$
- * Find polynomial THAT PASSES THRU ALL POINTS

$$y(x) \approx q_0 + q_1 x + q_2 x^2 + \dots + q_n x^n$$

FITTED:

- Thm: For $n+1$ points $\exists$ exactly one $\deg(n)$ polynomial THAT PASSES THRU all
- Error in interpolation $\sim (\Delta x)^n$

⇒ For problem sketched above, a 4th order polynomial
WILL INTERPOLATE ALL POINTS

$$\exists (a_0, a_1, a_2, a_3, a_4)$$

$$\text{S.T. } a_0 + a_1 x_i + a_2 x_i^2 + a_3 x_i^3 + a_4 x_i^4 = y_i \quad \text{for all } i$$

In Matrix Form

$$\begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 & x_1^4 \\ 1 & x_2 & x_2^2 & x_2^3 & x_2^4 \\ 1 & x_3 & x_3^2 & x_3^3 & x_3^4 \\ 1 & x_4 & x_4^2 & x_4^3 & x_4^4 \\ 1 & x_5 & x_5^2 & x_5^3 & x_5^4 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{pmatrix}$$

"Vandermonde Matrix" $\tilde{V}$

↳ Input $\tilde{V}$ with `numpy.linalg.inv`

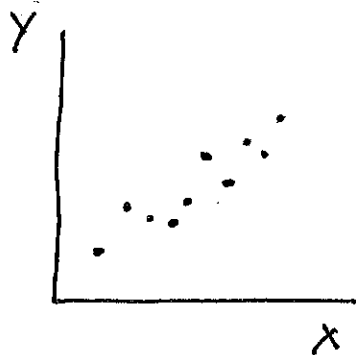
$$\vec{a} = \tilde{V}^{-1} \vec{y}$$

<See Vandermonde.ipynb>

⇒ Now... all the above assumes $\tilde{A}$ is invertible
 Under what conditions is this true?

- $\det(\tilde{A}) \neq 0$ {rows linearly independent}
- $\tilde{A}$ is square Cramer $\tilde{A}^{-1} = \frac{C(\tilde{A})^T}{\det(\tilde{A})}$

⇒ WHAT IF $\tilde{A}$ is NOT square? e.g. THE FITTING PROBLEM...



Model predicts $y = a_0 + a_1 x$
 i.e. linear relation between
 points

In matrix form

$$\begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \end{pmatrix}$$

WHAT IS THE
 INVERSE OF THIS?

⇒ Generalized Approach: Singular Value Decomposition

$$\begin{array}{ccccccc} \text{Thm} & \tilde{A} & = & \tilde{U} & \tilde{S} & \tilde{V}^+ \\ & \uparrow & & \uparrow & \uparrow & \nwarrow \\ & m \times n & & m \times m & m \times n & n \times n \\ & & & \text{"UNITARY"} & \text{POSITIVE DIAGONAL} & \text{"UNITARY"} \end{array}$$

UNITARY := Inverse = conjugate Transpose

$$\tilde{U}^{-1} = (\tilde{U}^*)^T = \tilde{U}^+$$

and $\tilde{A}^{-1} = \tilde{V} \tilde{S}^{-1} \tilde{U}^+$

Diagonals of $\tilde{S}$ called "singular values"

If all SV $\neq 0$, $\exists \tilde{S}^{-1}$

⇒ Numerical issue: IF any SV $\ll$ others, result will be loss of precision

$$\text{"Condition Number"} = \frac{\text{largest SV}}{\text{Smallest SV}}$$

IF CN $\gg 1$, matrix is "ill conditioned"

⇒ Strategy - set inverse of small SV = 0

Thus compute "pseudoinverse" (also called "generalized inverse")

↳ Best possible for ill-conditioned matrix.

<NOTEBOOK : Pseudoinverse.ipynb>

Lab (Due Today) - SLIGHT VARIATION ON NOTEBOOK

HW01 (Due 9/7) - Explicit calculation of pseudoinverse
By SVD & "QR" Methods

↳ `numpy.linalg.svd`
`numpy.linalg.qr`